

SKILL WEEK – 7

Session – 13

TASK 1: To parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir -p ~/Week7
cd ~/Week7
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ nano session13_task1.c
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ echo -e "Line 1\nLine 2\nLine 3\nLine 4" > input.txt
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ gcc session13_task1.c -o session13_task1
./session13_task1
Session 11 - Input Redirection
Enter command with input redirection:
myshell> wc < input.txt

Command      : wc
Input file   : input.txt
Redirection  : stdin < file

Input file opened successfully.
Executing command with redirected stdin...
4

Input redirection completed successfully.
File descriptor cleaned up.
Parent process restored normal stdin.
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ |
```

```
GNU nano 8.7.1 session13_task1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200

int main(void) {
    char input[MAX_INPUT];
    char command[100];
    char filename[100];

    printf("Session 11 - Input Redirection\n");
    printf("Enter command with input redirection:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    /* Parse input redirection */
    char *redirect = strchr(input, '<');
    if (redirect == NULL) {
        printf("No input redirection found.\n");
        return 1;
    }

    *redirect = '\0';
    strcpy(command, input);

    /* Remove spaces before filename */
    char *file = redirect + 1;
    while (*file == ' ' || *file == '\t')
        file++;

    strcpy(filename, file);

    if (strlen(filename) == 0) {
        printf("Error: Missing input file.\n");
    }
}
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session13_task1.c
GNU nano 8.7.1
if (strlen(filename) == 0) {
    printf("Error: Missing input file.\n");
    return 1;
}

printf("\nCommand      : %s\n", command);
printf("Input file     : %s\n", filename);
printf("Redirection      : stdin < file\n");

/* Open input file */
int fd = open(filename, O_RDONLY);
if (fd == -1) {
    perror("Error opening input file");
    return 1;
}

printf("\nInput file opened successfully.\n");

pid_t pid = fork();
if (pid < 0) {
    perror("fork failed");
    close(fd);
    return 1;
}

if (pid == 0) {
    /* Redirect stdin to file */
    if (dup2(fd, STDIN_FILENO) == -1) {
        perror("dup2 failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    close(fd);

    printf("Executing command with redirected stdin...\n");
    execlp("wc", "wc", "-l", NULL);
    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Parent closes file descriptor */
close(fd);
```

```
if (pid == 0) {
    /* Redirect stdin to file */
    if (dup2(fd, STDIN_FILENO) == -1) {
        perror("dup2 failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    close(fd);

    printf("Executing command with redirected stdin...\n");
    execlp("wc", "wc", "-l", NULL);
    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Parent closes file descriptor */
close(fd);

/* Wait for child */
waitpid(pid, NULL, 0);

printf("\nInput redirection completed successfully.\n");
printf("File descriptor cleaned up.\n");
printf("Parent process restored normal stdin.\n");

return 0;
}
```

TASK 2: To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

```
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ nano session13_task2.c
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ gcc session13_task2.c -o session13_task2
./session13_task2
Session 13 - Output Redirection
Enter command with output redirection:
myshell> echo hello > output.txt

Command      : echo hello
Output file   : output.txt
Redirection   : stdout > file

Output file opened successfully.

Output redirection completed successfully.
File descriptor cleaned up.
Parent process restored normal stdout.

Verified output data:
Output redirected successfully

Output verification completed successfully.
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ |
```

```
GNU nano 8.7.1 session13_task2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200

int main(void) {
    char input[MAX_INPUT];
    char command[100];
    char filename[100];

    printf("Session 13 - Output Redirection\n");
    printf("Enter command with output redirection:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    /* Find output redirection symbol > */
    char *redirect = strchr(input, '>');
    if (redirect == NULL) {
        printf("No output redirection found.\n");
        return 1;
    }

    *redirect = '\0';
    strcpy(command, input);

    /* Remove spaces before filename */
    char *file = redirect + 1;
    while (*file == ' ' || *file == '\t')
        file++;

    strcpy(filename, file);

    if (strlen(filename) == 0) {
        printf("Error: Missing output file.\n");
    }
}
```

```
GNU nano 8.7.1 session13_task2.c
if (strlen(filename) == 0) {
    printf("Error: Missing output file.\n");
    return 1;
}

printf("\nCommand      : %s\n", command);
printf("Output file     : %s\n", filename);
printf("Redirection       : stdout > file\n");

/* Create or truncate output file */
int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1) {
    perror("Error opening output file");
    return 1;
}

printf("\nOutput file opened successfully.\n");

pid_t pid = fork();
if (pid < 0) {
    perror("fork failed");
    close(fd);
    return 1;
}

if (pid == 0) {
    /* Redirect stdout to output file */
    if (dup2(fd, STDOUT_FILENO) == -1) {
        perror("dup2 failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    close(fd);

    execlp("echo", "echo", "Output redirected successfully", NULL);
    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Parent closes its copy of the descriptor */
close(fd);

/* Wait for child */
}
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session13_task2.c
GNU nano 8.7.1
if (pid == 0) {
    /* Redirect stdout to output file */
    if (dup2(fd, STDOUT_FILENO) == -1) {
        perror("dup2 failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    close(fd);

    execlp("echo", "echo", "Output redirected successfully", NULL);
    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Parent closes its copy of the descriptor */
close(fd);

/* Wait for child */
waitpid(pid, NULL, 0);

printf("\nOutput redirection completed successfully.\n");
printf("File descriptor cleaned up.\n");
printf("Parent process restored normal stdout.\n");

/* Verify output data */
FILE *fp = fopen(filename, "r");
if (fp == NULL) {
    perror("Error verifying output file");
    return 1;
}

char buffer[200];
printf("\nVerified output data:\n");
while (fgets(buffer, sizeof(buffer), fp) != NULL) {
    printf("%s", buffer);
}
fclose(fp);

printf("\nOutput verification completed successfully.\n");
return 0;
}
```

Session – 14

TASK 1: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

```
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ nano session14_task1.c
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ echo "Output redirected successfully" > output.txt
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ gcc session14_task1.c -o session14_task1
./session14_task1
Session 13 - Append Redirection
Enter command with append redirection:
myshell> echo hello >> output.txt

Command      : echo hello
Output file   : output.txt
Redirection   : stdout >> file

Output file opened in append mode.

Append operation completed successfully.
Existing data was preserved.
File descriptor cleaned up.

File contents after append:
Output redirected successfully
New data appended successfully

Append validation completed successfully.
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ |
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session14_task1.c
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200

int main(void) {
    char input[MAX_INPUT];
    char command[100];
    char filename[100];

    printf("Session 13 - Append Redirection\n");
    printf("Enter command with append redirection:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    /* Find append operator >> */
    char *redirect = strstr(input, ">>");
    if (redirect == NULL) {
        printf("No append redirection found.\n");
        return 1;
    }

    *redirect = '\0';
    strcpy(command, input);

    /* Get filename */
    char *file = redirect + 2;
    while (*file == ' ' || *file == '\t')
        file++;

    strcpy(filename, file);

    if (strlen(filename) == 0) {
        printf("Error: Missing output file.\n");
    }
}
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session14_task1.c
GNU nano 8.7.1
    if (strlen(filename) == 0) {
        printf("Error: Missing output file.\n");
        return 1;
    }

    printf("\nCommand      : %s\n", command);
    printf("Output file       : %s\n", filename);
    printf("Redirection        : stdout >> file\n");

    /* Open file in append mode */
    int fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd == -1) {
        perror("Error opening output file");
        return 1;
    }

    printf("\nOutput file opened in append mode.\n");

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        /* Redirect stdout to file */
        if (dup2(fd, STDOUT_FILENO) == -1) {
            perror("dup2 failed");
            close(fd);
            exit(EXIT_FAILURE);
        }
        close(fd);

        execlp("echo", "echo", "New data appended successfully", NULL);
        perror("exec failed");
        exit(EXIT_FAILURE);
    }

    /* Parent closes descriptor */
    close(fd);

    /* Wait for child */
}
```

```
GNU nano 8.7.1 session14_task1.c

if (pid == 0) {
    /* Redirect stdout to file */
    if (dup2(fd, STDOUT_FILENO) == -1) {
        perror("dup2 failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    close(fd);

    execlp("echo", "echo", "New data appended successfully", NULL);
    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Parent closes descriptor */
close(fd);

/* Wait for child */
waitpid(pid, NULL, 0);

printf("\nAppend operation completed successfully.\n");
printf("Existing data was preserved.\n");
printf("File descriptor cleaned up.\n");

/* Verify appended data */
FILE *fp = fopen(filename, "r");
if (fp == NULL) {
    perror("Error verifying output file");
    return 1;
}

char buffer[200];
printf("\nFile contents after append:\n");
while (fgets(buffer, sizeof(buffer), fp) != NULL) {
    printf("%s", buffer);
}
fclose(fp);

printf("\nAppend validation completed successfully.\n");
return 0;
}
```

TASK 2 : TO parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

```
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ nano session14_task2.c
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ gcc session14_task2.c -o session14_task2
./session14_task2
Session 13 - stderr Redirection
Enter command with error redirection:
myshell> ls /nonexistent_directory 2> error.txt

Command      : ls /nonexistent_directory
Error file    : error.txt
Redirection   : stderr 2> file

Error output file opened successfully.

Error stream redirection completed.
Error output captured in file.
Child exit status: 2

Captured error output:
ls: cannot access '/nonexistent_directory': No such file or directory

stderr redirection validation completed successfully.
shreyaj@LAPTOP-FT6VVM4D:~/Week7$ |
```

```
GNU nano 8.7.1 session14_task2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200

int main(void) {
    char input[MAX_INPUT];
    char command[100];
    char filename[100];

    printf("Session 13 - stderr Redirection\n");
    printf("Enter command with error redirection:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    /* Find stderr redirection operator 2> */
    char *redirect = strstr(input, "2>");
    if (redirect == NULL) {
        printf("No stderr redirection found.\n");
        return 1;
    }

    *redirect = '\0';
    strcpy(command, input);

    /* Get output filename */
    char *file = redirect + 2;
    while (*file == ' ' || *file == '\t')
        file++;

    strcpy(filename, file);

    if (strlen(filename) == 0) {
        printf("Error: Missing error output file.\n");
        return 1;
    }
}
```

```
GNU nano 8.7.1 session14_task2.c
if (strlen(filename) == 0) {
    printf("Error: Missing error output file.\n");
    return 1;
}

printf("\nCommand      : %s\n", command);
printf("Error file       : %s\n", filename);
printf("Redirection       : stderr 2> file\n");

/* Open file for stderr */
int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1) {
    perror("Error opening error file");
    return 1;
}

printf("\nError output file opened successfully.\n");

pid_t pid = fork();
if (pid < 0) {
    perror("fork failed");
    close(fd);
    return 1;
}

if (pid == 0) {
    /* Redirect stderr to file */
    if (dup2(fd, STDERR_FILENO) == -1) {
        perror("dup2 failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    close(fd);

    /* Generate an error intentionally */
    execlp("ls", "ls", "/nonexistent_directory", NULL);
    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Parent closes descriptor */
close(fd);
}
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session14_task2.c
GNU nano 8.7.1
    if (dup2(fd, STDERR_FILENO) == -1) {
        perror("dup2 failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    close(fd);

    /* Generate an error intentionally */
    execlp("ls", "ls", "/nonexistent_directory", NULL);
    perror("exec failed");
    exit(EXIT_FAILURE);
}

/* Parent closes descriptor */
close(fd);

/* Wait for child */
int status;
waitpid(pid, &status, 0);

printf("\nError stream redirection completed.\n");
printf("Error output captured in file.\n");
printf("Child exit status: %d\n", WEXITSTATUS(status));

/* Verify captured error output */
FILE *fp = fopen(filename, "r");
if (fp == NULL) {
    perror("Error verifying error file");
    return 1;
}

char buffer[256];
printf("\nCaptured error output:\n");
while (fgets(buffer, sizeof(buffer), fp) != NULL) {
    printf("%s", buffer);
}
fclose(fp);

printf("\nstderr redirection validation completed successfully.\n");
return 0;
}
```